ASSINGMENT 3: Views and Indexes (SQLite Edition)
ATHANASIOS GOURDOMICHALIS

1. Objective

This assignment focuses on utilizing Views to simplify complex queries and studying Indexes to optimize performance, exclusively within the SQLite environment.

2. Prerequisites

The pre-existing database schema is assumed to be the one designed in the 1st Assignment (not the 2nd). All answers and explanations must be provided as SQL comments (using /* ... */) within the respective script files to ensure that they remain fully executable.

Part A: Virtual Tables (Views)

3. Objective 1. View Creation: FullTrackInfo

- We must create a virtual table (View) named **FullTrackInfo**. This view must join tracks with their corresponding albums and artists. **The goal is** to simplify data retrieval by avoiding writing multiple JOIN clauses in every query.

4. Objective 2. View Utilization

- We must write an SQL query that uses the **FullTrackInfo** view (e.g., retrieving all tracks by a specific artist, ordered by album name in alphabetical order).

Part B: Index Analysis

5. Objective 3. Inspecting Indexes in SQLite

We must use **PRAGMA** commands to examine the **r_albums_artists** and **albums** tables:

- We must execute **PRAGMA index_list('table_name');** to retrieve the names of the indexes defined on both tables.
- Subsequently, we must use **PRAGMA index_info('index_name');** to inspect the structure and columns associated with each index.

6. Objective 4. Automatic Index Generation Analysis (Commentary)

- We must examine the outputs of the preceding commands. What do you observe regarding SQLite's behavior concerning the automatic creation of indexes when Primary Keys or Unique constraints are defined? We must provide our analysis in the form of a comment block inside the b.sql file.

Part C: Query Optimization and Execution Plans

7. Objective 5. Execution Plan Analysis

- We must examine the query execution plan for the query below using the **EXPLAIN QUERY PLAN** command:

```
SELECT t.name
FROM tracks t
JOIN r_track_artist rta ON t.id = rta.track_id
JOIN artists a ON rta.artist_id = a.id
WHERE a.name = 'Ludwig van Beethoven';
```

- Next, we must create two indices on the **artists** and **r_track_artist** tables to optimize performance, and evaluate the updated execution plan by using **EXPLAIN QUERY PLAN** once more. We have to verify that our newly created indices are actively used within the generated execution plan.

8. Note: SQLite Execution Plan Terminology Definitions:

- **SCAN TABLE: Indicates a full table scan (slow and inefficient for large datasets).**

- **SEARCH TABLE: Indicates the use of an index structure for targeted record lookup (fast and efficient).**

- **USING COVERING INDEX: Indicates that the query was answered entirely by scanning the index structure alone, without fetching data blocks from the underlying table (optimal performance).**

9. Project Files

- ***a.sql:*** Contains the DDL statements for creating the View (Query 1) and the DML query utilizing it (Query 2).

- ***b.sql:*** Contains the **PRAGMA** commands (Query 3) along with the analysis on automatic index generation formatted as SQL comments (Query 4).

- ***c.sql:*** Contains the CREATE INDEX DDL commands and the corresponding EXPLAIN QUERY PLAN commands (Query 5).